

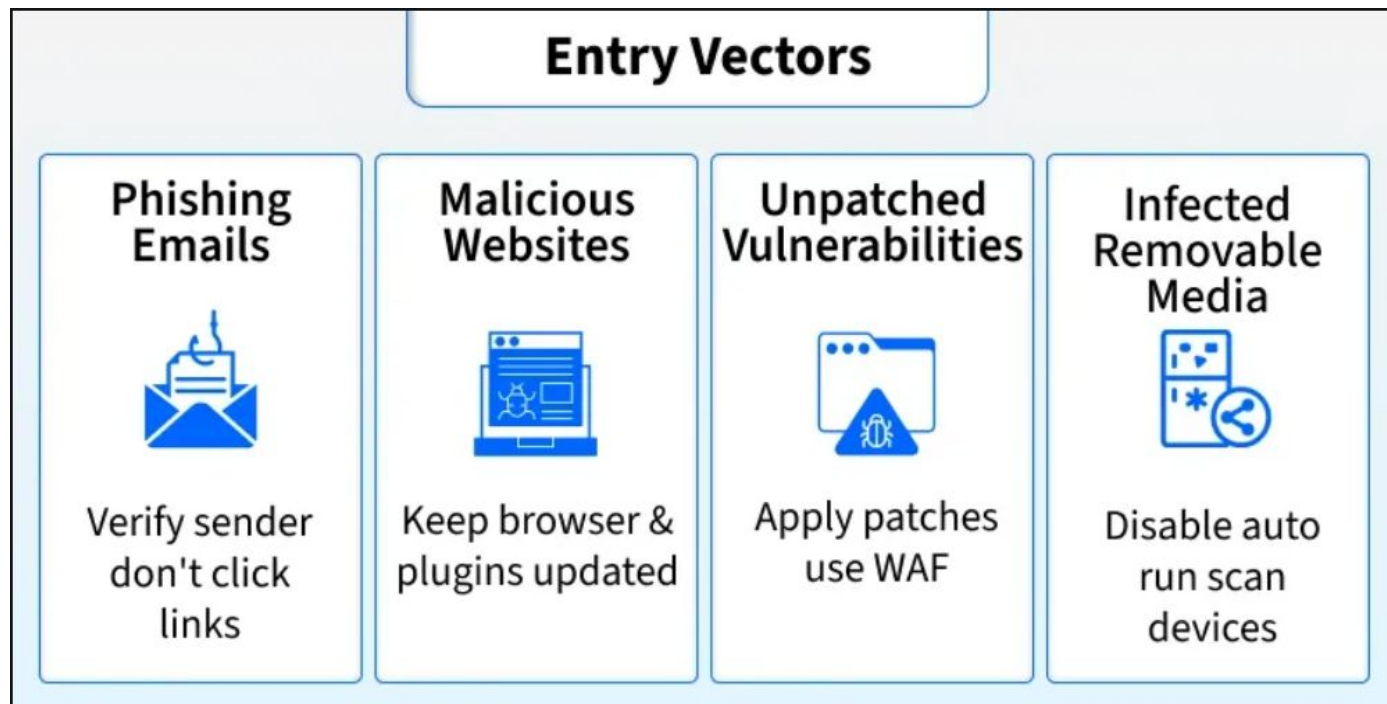
System & Software Security

Chapter II

Malware

- Malware is software that infects systems without user consent to steal sensitive data (bank details, passwords, personal emails), disrupt operations, or alter core system behavior.
- It can exfiltrate confidential information, corrupt or delete files, and impair system availability or integrity.

- Mitigate risks with up-to-date antivirus and caution when opening links or attachments.
- Malware commonly spreads through the following vectors:



Types of Malware



- **Viruses** - A Virus is a malicious executable code attached to another executable file.
- The virus spreads when an infected file is passed from system to system.
- Viruses can be harmless or they can modify or delete data. Opening a file can trigger a virus.
- Once a program virus is active, it will infect other programs on the computer.

- **Worms** - Worms replicate themselves on the system, attaching themselves to different files and looking for pathways between computers, such as computer network that shares common file storage areas.
- Worms usually slow down networks.
- A virus needs a host program to run but worms can run by themselves.
- After a worm affects a host, it is able to spread very quickly over the network.

- **Trojan horse** - A [Trojan horse](#) is malware that carries out malicious operations under the appearance of a desired operation such as playing an online game.
- A Trojan horse varies from a virus because the Trojan binds itself to non-executable files, such as image files, and audio files.

- **Ransomware** - Ransomware grasps a computer system or the data it contains until the victim makes a payment.
- **Ransomware** encrypts data in the computer with a key that is unknown to the user.
- The user has to pay a ransom (price) to the criminals to retrieve data. Once the amount is paid the victim can resume using his/her system.

- **Adware** - It displays unwanted ads and pop-ups on the computer.
- It comes along with software downloads and packages.
- It generates revenue for the software distributor by displaying ads.
- **Spyware** - Its purpose is to steal private information from a computer system for a third party.
- [Spyware](#) collects information and sends it to the hacker.

- **Logic Bombs** - A logic bomb is a malicious program that uses a trigger to activate the malicious code.
- The logic bomb remains non-functioning until that trigger event happens.
- Once triggered, a logic bomb implements a malicious code that causes harm to a computer.
- Cybersecurity specialists recently discovered logic bombs that attack and destroy the hardware components in a workstation or server including the cooling fans, hard drives, and power supplies.
- The logic bomb overdrives these devices until they overheat or fail.

- **Rootkits** - A [rootkit](#) modifies the OS to make a backdoor.
- Attackers then use the backdoor to access the computer distantly.
- Most rootkits take advantage of software vulnerabilities to modify system files.
- **Backdoors** - A backdoor bypasses the usual authentication used to access a system.
- The purpose of the backdoor is to grant cyber criminals future access to the system even if the organization fixes the original vulnerability used to attack the system.

- **Keyloggers** - Keylogger records everything the user types on his/her computer system to obtain passwords and other sensitive information and send them to the source of the keylogging program.

Anti-Malware Strategies

- **Proactive Prevention:** Regularly updating software and operating systems to patch vulnerabilities.
- **Defense-in-Depth:** Implementing multiple layers of security, such as firewalls, intrusion detection systems (IDS), and web filtering.
- **Zero Trust Architecture:** Adopting a strict, least-privilege access policy to restrict malware movement.
- **User Training:** Educating users to avoid suspicious links and email attachments.
- **Data Backups:** Maintaining regular, offline backups to restore data in case of ransomware.
- **Network Segmentation:** Dividing networks into smaller, isolated zones to contain potential breaches.

Anti-Malware Tools

- **Antivirus/Anti-Malware Software:** Tools like Malwarebytes, [Microsoft Defender](#), and others provide signature-based detection and real-time scanning.
- **Endpoint Detection and Response (EDR):** Advanced tools for monitoring, detecting, and responding to threats at the endpoint level.
- **Network Security Tools:** Firewalls and network traffic analyzers, such as Wireshark and [Fiddler](#), help detect unusual traffic.
- **Email & Web Filtering:** Solutions that block malicious websites and phishing emails.
- **Deception Technology:** Using honeypots and decoys to lure and analyze attackers.
- **Memory Protection Tools:** Defend against fileless malware that operates in memory.

Unintentional (Non-malicious) Programming flaws

- Unintentional or non-malicious programming flaws are accidental software bugs, logic errors, or oversights during design and coding that create security vulnerabilities.
- These common mistakes, such as buffer overflows, improper input validation, and race conditions, do not stem from malicious intent but can lead to severe system instability, data breaches, or arbitrary code execution.

Key Types of Non-Malicious Programming Flaws

- **Buffer Overflows**: Occur when a program writes more data to a storage area (buffer) than it can hold, overwriting adjacent memory. This can lead to system crashes or arbitrary code execution.
- **Input Validation Errors (Incomplete Mediation)**: Failures to properly sanitize user input, allowing attackers to manipulate input fields to execute unauthorized actions, such as SQL injection or Cross-Site Scripting (XSS).
- **Race Conditions (TOCTOU - Time-of-check to time-of-use)**: A flaw where a program checks a resource's status (like file permissions) but that status changes before the program acts on it, allowing for unauthorized access.
- **Memory Leaks/Uninitialized Variables**: Errors in memory management where applications fail to release unused memory or use variables before assigning them, potentially leading to system instability.
- **Integer Overflow/Underflow**: Calculations exceeding the maximum or minimum value allowed for an integer type, causing unexpected behavior in loops or memory allocation.
- **Security Misconfigurations**: Leaving default passwords, unnecessary services enabled, or, improper, overly permissive access rights.

Buffer Overflow

- Buffers are used to store specific amounts of information.
- Unless the program utilizing the buffer has built-in instructions to discard data when too much is sent, the data in memory adjacent to the buffer will be overwritten.
- Attackers can take advantage of buffer overflows to corrupt software.
- Buffer overflow attacks, despite being well-understood, remain a serious security issue that plagues cyber-security teams.
- Buffer overflows are classified based on the buffer's location in the process memory.
- Stack-based or heap-based overflows are the most common.
- Both are stored in the random access memory of a device.

Stack-based Buffer Overflow

- A stack, has a fixed size.
- The code developer must set aside a certain amount of space for the stack.
- The stack will overflow if the user's input is longer than the amount of space designated within the stack and the software does not verify that the input will fit.
- This isn't a significant issue in and of itself, but when combined with malicious input, it becomes a major security flaw.

- **Heap overflow attack** - This type of attack targets data in the open memory pool known as the heap*.
- **Integer overflow attack** - In an integer overflow, an arithmetic operation results in an integer (whole number) that is too large for the integer type meant to store it; this can result in a buffer overflow.

Incomplete mediation

- **Definition:** Incomplete mediation refers to the failure of a system to enforce security policies consistently across all entry points. It involves verifying that a subject is authorized to perform an operation on an object, but failing to do so completely can lead to security vulnerabilities.
- **Impact:** This can lead to vulnerabilities where attackers exploit the system by bypassing security checks, resulting in unauthorized access or data corruption. For instance, if data is changed between the access check and its use, it can lead to serious security implications.

Examples:

- **Input Validation:** If a web application does not validate user input on all pages, an attacker might submit malicious data that the system accepts. This is a common example where incomplete mediation can lead to errors and potential exploitation.
- **Session Management:** If a user's session is not properly validated after login, an attacker could hijack the session and gain unauthorized access. This highlights the importance of consistent mediation to prevent unauthorized actions.

- <http://site.com/buy?item=1&price=100>
- <http://site.com/buy?item=1&price=1>

Prevention Strategies

- **Consistent Input Validation:** Ensure that all user inputs are validated at every entry point. This involves validating all input to prevent unauthorized data manipulation.
- **Access Controls:** Implement strict access controls to verify user permissions before allowing actions. This includes complete mediation using a reference monitor to ensure that every requested access is governed by an access policy.
- **Regular Security Audits:** Conduct regular audits and testing to identify and fix potential vulnerabilities. Testing program components, such as module and integration testing, can help ensure that all parts of the system function correctly and securely.

Time of-Check to Time-of-Use

- Time of-Check to Time-of-Use is a race condition security vulnerability occurring when a system verifies a condition (e.g., file permissions, file existence) but the resource changes before the system uses it.
- An attacker manipulates the resource during this gap, leading to unauthorized access, privilege escalation, or data corruption.

Key Aspects of TOCTOU

- **The Vulnerability Window:** A small, critical time interval exists between the check and the actual use, allowing for malicious intervention.
- **Examples:** A program checks if a file exists (safe), but before it writes to the file, an attacker replaces it with a symbolic link to a system file, causing the program to overwrite critical data.
- **Common Occurrences:** Often found in file system operations (Unix), local sockets, or database transactions.
- **Mitigation:** The most effective defense is to make the operation atomic (combining check and use) or by using file descriptors instead of file names, which ensures the resource cannot be replaced between check and use.

Classic File System Example:

- **Check:** A privileged (setuid) program checks if `/home/user/log` is owned by the user (safe to write).
- **Race:** The attacker quickly deletes `/home/user/log` and creates a symbolic link: `ln -s /etc/passwd /home/user/log`.
- **Use:** The program opens `/home/user/log`, but now it is writing to `/etc/passwd`.

Undocumented access points

- Undocumented access points, often referred to as "rogue access points" or "hidden interfaces, are network devices or software functionalities that operate without authorization or public knowledge, posing significant security risks.
- These access points can serve as entry points for attackers to bypass existing security controls, facilitate lateral movement, or establish persistence within a network.

Key Aspects of Undocumented Access Points and Interfaces:

- **Rogue Access Points:** These are unauthorized, often unmanaged, Wi-Fi access points connected to a secure network. They can create an "open door" for attackers to access sensitive information, bypassing traditional perimeter defenses.
- **Hidden Features and Backdoors:** Manufacturers may include hidden functionality, sometimes called "chicken bits," for testing or maintenance purposes. These interfaces, if not properly secured, can be exploited to read/write to memory, execute arbitrary code, or provide undocumented access to systems.

Unterminated Null Terminated String

- An unterminated null occurs when a character array intended to be a null-terminated string lacks the `\0` character at its logical end.
- This is a common programming error with potentially serious consequences.

Consequences:

- **Buffer Overflows:** The function might write data past the end of the allocated buffer, overwriting other data in memory.
- **Security Vulnerabilities:** String termination errors are a common source of security problems.
- **Crashes:** The program may terminate unexpectedly with an error like a segmentation fault or access violation.
- **Logic Errors:** The program may output unexpected or "garbage" data from adjacent memory.

Parameter length

- Parameter length refers to enforcing strict minimum and maximum byte limits on input data (URL parameters, form fields) to prevent attacks like buffer overflows and SQL injection.
- It validates that incoming data adheres to defined constraints, with violations indicating potential malicious activity.

Key aspects include:

- **Buffer Overflow Protection:** Excessively long parameters can exploit weak application handling, causing system crashes or unauthorized code execution.
- **Input Validation:** Security policies (e.g., in Web Application Firewalls) check that input lengths match expected, defined limits.
- **Security Parameter Index (SPI):** In networking (IPsec), this is a 32-bit identifier used to link packets to a specific security association.
- **Setting Limits:** Best practices involve setting appropriate maximum lengths to prevent attackers from using long inputs to bypass security mechanisms or hide malicious payloads.
- **Password Policies:** Parameters also define minimum character lengths and complexity for user credentials.

Unsafe Utility Programs

- Programs branded as "utilities" can sometimes be unsafe due to malicious intent, excessive bundling of adware, or destructive actions.
- **Trojan Horse Utilities:** Malicious programs disguised as useful tools (e.g., fake antivirus or system cleaners) that compromise data or steal information.
- **Rogue Registry Cleaners/Optimizers:** Software that claims to fix hundreds of "errors" to scare users into buying the full version, sometimes causing system instability.
- **Bundled Adware/Spyware:** Free utilities that install additional, unwanted software.

Race Condition

- https://youtu.be/XH_KVNGsKpA

Countermeasures for Users

- Use software acquired from reliable sources
- Test software in an isolated environment
- Only open attachments when you know them to be safe
- Treat every website as potentially harmful
- Create and maintain backups

Countermeasures for Developers

- Software Engineering Techniques
- Information hiding - Information hiding: describing what a module does, not how
- Modularity – Coupling and cohesion
- Mutual Suspicion
- Confinement - possible damage does not spread to other parts of a system
- Simplicity - Complexity is often the enemy of security
- Genetic Diversity - Diversity reduces the number of targets susceptible to one attack type
- Testing

Code Testing

- Unit testing
- Integration testing
- Function testing
- Performance testing
- Acceptance testing
- Installation testing
- Regression testing
- Penetration testing

Countermeasure Specifically for Security –

Design Principles for Security

- Least privilege
- Economy of mechanism
- Open design
- Complete mediation
- Permission based
- Separation of privilege
- Least common mechanism
- Ease of use

Countermeasure Specifically for Security –

Design Principles for Security

- Penetration Testing for Security
- Proofs of Program Correctness
- Validation
- Defensive Programming - Program designers and implementers need not only write correct code but must also anticipate what could go wrong.
- Trustworthy Computing Initiative - all developers undergo security training, and secure software development practices were instituted throughout the company

Countermeasures that Don't Work

- **Penetrate-and-Patch** - Penetrate-and-patch fails because it is hurried, misses the context of the fault, and focuses on one failure, not the complete system.
- **Security by Obscurity** - Security by obscurity is the belief that a system can be secure as long as nobody outside its implementation group is told anything about its internal mechanisms.

Secure Software Development Lifecycle (SSDLC)



Key Principles of Secure SDLC



Security by Design

integrates security requirements early in software development planning and design.



Continuous Monitoring

security checks for ongoing issue resolution.



Risk Assessment

Identify, prioritize, and mitigate security risks.



Education and Training

Secure software Developers need comprehensive training for effective issue resolution.



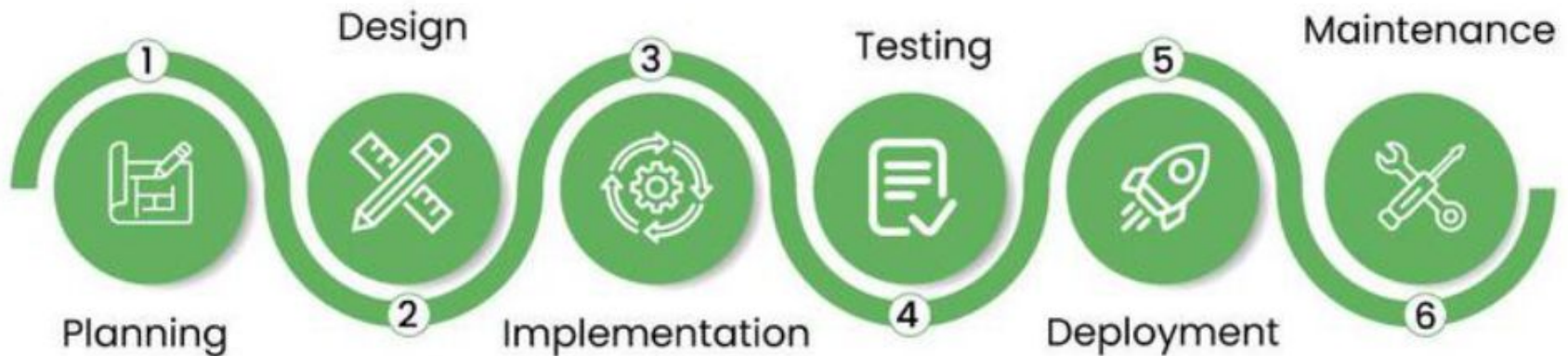
Collaboration

Collaboration in software development encourages teamwork for common security goals through knowledge sharing.

- **Security by Design:** This principle highlights the need to think about security right from the start of creating software. It means including security requirements in the initial planning and design stages of development.
- **Continuous Monitoring:** Continuous monitoring is an ongoing process of regularly checking and ensuring the security of the software at every step of its development. It's not a one-time thing but a continuous effort to find and fix security issues throughout the entire development process.
- **Risk Assessment:** This principle involves evaluating and understanding potential security risks early in the development process. It includes identifying vulnerabilities and deciding which risks need the most attention and quick action to reduce potential threats.

- **Education and Training:** Education and training are essential to make sure everyone involved in creating the software, especially developers, knows about security issues. It emphasizes providing the necessary training so that individuals have the skills to handle security concerns effectively.
- **Collaboration:** Collaboration is a crucial principle that stresses the importance of teamwork. It involves encouraging collaboration between different teams, like developers, operations, and security teams. This ensures that everyone is working together, sharing knowledge, and coordinating efforts to achieve common security goals during the software development process.

Phases of SSDLC



- **Planning:** In the planning stage, the main focus is on figuring out the security requirements for the software. This includes identifying possible risks and creating a plan for how to make the software secure from the beginning.
- **Design:** During the design phase, the plan for security is put into action. This involves making decisions about how to build security features into the software. The goal is to ensure that the design can handle potential security problems.
- **Implementation:** In the implementation phase, developers start building the software using secure coding practices. This means writing code in a way that reduces the chances of security problems. Code reviews are done to catch and fix any security issues.

- **Testing:** Testing is all about checking how secure the software is. Different tests are done, like trying to break into the software to find vulnerabilities, scanning the code for potential problems, and making sure the software can handle different security threats.
- **Deployment:** Deployment is when the software is released. Here, the focus is on making sure the release process is secure, taking precautions to avoid any security issues during this stage.
- **Maintenance:** Maintenance is an ongoing process where the software is continuously looked after. This involves keeping an eye on security and regularly updating the software to deal with new threats, making sure it stays secure over time.

SDLC Process vs SSDLC Process:



Aspect	Software Development Life Cycle (SDLC)	Secure Software Development Life Cycle (SSDLC)
Focus	Primarily on delivering functional software.	Emphasizes security considerations throughout the process.
Objective	Meeting business requirements and functionality.	Integrating security measures to protect against vulnerabilities.
Phases	Typically includes phases like planning, analysis, design, implementation, testing, deployment, and maintenance.	Incorporates security-related phases, such as threat modeling, secure coding, and continuous monitoring.
Security Considerations	Security is addressed mainly during the testing phase.	Security is considered at every stage, from design to deployment.
Testing	Security testing is often a separate phase.	Security testing is integrated into each phase of development.
Documentation	Documentation focuses on functionality and features.	Comprehensive documentation on security measures, threat models, and risk assessments.
Development Speed	Emphasizes timely delivery of software features.	May involve slightly longer development cycles due to added security considerations.
Adaptability	Generally more adaptable to changes in business requirements.	May require more effort to incorporate changes due to the stringent security focus.

Privacy By Design

- Privacy by Design (PbD) is an approach for the design and development of a digital solution that requires privacy to be embedded right from the design stage and then throughout the development lifecycle so that privacy becomes an integral part of core functionality as opposed to an afterthought.



7 Principles of Privacy by Design



7 Principles of Privacy by Design

- Proactive not Reactive; Preventative not Remedial
- Privacy as the Default Setting
- Privacy Embedded into Design
- Full Functionality – Positive-Sum not Zero-Sum
- End-to-End Security – Full Lifecycle Protection
- Visibility and Transparency – Keep it Open
- Respect for User Privacy – Keep it User-Centric

Principle 1: Proactive not reactive; Preventative not remedial

- A privacy-first attitude supports a preventative approach to privacy.
- Instead of reacting to privacy risks or invasions when they happen, companies will actively build processes and procedures to prevent them from occurring in the first place.

Principle 2: Privacy as the default setting

- Users shouldn't have to worry about their privacy settings when browsing a website, opening an app, or logging into software.
- Privacy as Default ensures they don't have to.
- It automatically sets users' privacy to the highest level of protection, whether or not a user interacts with those settings.

- Such default settings include:
- **Collection limitation:** You only collect the amount and types of data you're legally allowed to.
- **Data minimization:** You collect only the absolute minimum amount of data necessary. You won't collect data just for the sake of collection or because you can.
- **Use, retention & disclosure limitation:** You won't use the collected data for any other purpose than to which the user has agreed. You won't keep data after it's no longer needed for the purposes you stated to users, and you won't disclose the data unless necessary to achieve the purpose for which it was collected.
- **Security:** You implement appropriate technical and organizational measures, such as encryption, to ensure the confidentiality, integrity, and availability of the personal data.

Principle 3: Privacy embedded into design

- Protecting users' data and privacy should now be a part of the conversation when building a website, a mobile app, or a software application.
- For embedded privacy to work, it can't just be a feature tacked on at the end. It also can't be obvious or awkwardly included so as to detract from the functionality of the program you're designing.
- Every decision and new process must be filtered through a privacy-first mindset to promote both functionality and privacy protection.

Principle 4: Full functionality – Positive-sum, not zero-sum

- A fatalistic attitude won't work with Privacy by Design.
- Those who argue trade-offs must be made with the user experience or with security protocols have a zero-sum attitude.
- Those who work to integrate privacy into every design element seamlessly take a positive-sum approach.
- These innovators will see their brands grow in a world where privacy is increasingly a market mover, not just an issue of legal compliance.

Principle 5: End-to-end security – Lifecycle protection

- From the point at which users provide personal data, to when it can be destroyed after serving its purpose — and everything in between — Privacy by Design ensures the security of this data through the processing lifecycle.
- This full lifecycle protection is where the interdisciplinary nature of Privacy by Design shines.
- It leans heavily on security best practices to provide end-to-end data protection.
- Security also ensures data remains confidential, true to its original form, and accessible during its time with the company.

Principle 6: Visibility and transparency

– Keep it open

- Openness with users about your privacy policies and procedures builds accountability and trust.
- Privacy by Design means documenting and communicating actions clearly, consistently, and transparently.
- It presents a shared attitude of privacy as a duty, and one your team takes seriously.
- That promise should be supported by an accessible and effective complaint submission and resolution process, as well as independent verification of your policies and promises to users.

Principle 7: Respect for user privacy – Keep it user-centric

- Respect for user privacy involves always having the users' privacy interests in mind and providing the necessary safeguards and features to protect such interests.
- This respect inspires every design decision and understands that the best user experience puts privacy first.
- This includes putting the power in the hands of the user to manage their own data and actively seeking their engagement in the process.

Thank you!